

MemoryAgentBench results — FactConsolidation, 6K to 262K

Kaleidoscope on the FactConsolidation task of [MemoryAgentBench](#) (Hu, Wang and McAuley, ICLR 2026), at four history lengths, against keyword search, no memory, and the whole history pasted into the prompt. 800 questions.

Read this before the numbers

Every arm uses the same reader: GPT-5.6 Luna, reasoning effort **high**, with a 10-token answer cap. The only thing that differs between arms is what reaches its prompt. So the comparison *within* this page is controlled for the reader. It is not controlled against published MemoryAgentBench results, which use other readers, and none are compared here.

Three further things, stated rather than adjusted for:

- **The full-context arm is self-scored.** It answered all 100 single-hop questions at 262K but stopped before MemoryAgentBench’s own end-of-run scoring, so its 85 comes from [score_partial.py](#), which states its one assumption at the top. Multi-hop was not run for that arm. Its score also drifted down as it ran — 90% after 60 questions, 88% after 76, 85% at 100 — which is why a running cell is never quoted as settled.
- **Not yet re-run from this repository.** These numbers came from this code before it moved here. The move changed file names, paths, imports and lint, not what it measures.
- **The dataset caps some scores.** FactConsolidation says the newest version of a fact wins, but scores some questions against an older one. That makes 2-3% of single-hop and 13-34% of multi-hop questions unanswerable by its own rule, for every arm.

At 262K tokens

	single-hop accuracy	reader tokens per question
entire history in the prompt	85	291,746
Kaleidoscope	90	1,195
BM25 keyword search	78	2,066
no memory	13	189

Kaleidoscope is 5 points more accurate than pasting in the whole history, at 0.4% of the tokens — 244 times fewer. It beats keyword search by 12 points on 42% fewer tokens.

The whole-history arm is not a ceiling. Its errors are confident wrong answers naming an *older* value of a fact that was later overwritten: shown every version, the reader sometimes picks a stale one. That is consistent with the errors; it has not been measured directly.

Accuracy at every length

Out of 100 questions per cell, single-hop / multi-hop:

history length	no memory	BM25	Kaleidoscope
6K	19 / 0	97 / 33	95 / 15
32K	26 / 0	91 / 13	95 / 8
64K	24 / 0	85 / 13	96 / 10
262K	13 / 2	78 / 11	90 / 10

As the history grows, keyword search falls from 97 to 78 on single-hop. Kaleidoscope holds between 95 and 90. How often the answer was among the ten results each system considered:

history length	Kaleidoscope	BM25
6K	100%	96.9%
32K	100%	93.8%
64K	100%	86.9%
262K	98%	83.8%

Tokens at every length

Reader input tokens per single-hop question:

history length	no memory	BM25	Kaleidoscope	Kaleidoscope saves vs BM25
6K	189	1,879	854	55%
32K	190	1,999	948	53%
64K	189	2,007	1,019	49%
262K	189	2,066	1,195	42%

BM25 always sends ten chunks. Kaleidoscope sends what the question needs, which averages four to five.

Where Kaleidoscope loses

Multi-hop, at every length. BM25 leads 33 to 15 at 6K and stays ahead at every longer length. This is a real loss, not noise.

At shorter histories the second fact in a chain is almost always retrievable, so the failure there is in which results are kept and in what order rather than whether the store can find them. At 262K, reaching it is itself the problem. A second retrieval keyed on the first answer closes over half the chains; under the same conditions, keyword search closes more of them.

An earlier analysis reported a much higher closure rate. That probe took its bridge from the answer key, which no retriever has at question time. It was an oracle and is withdrawn.

Reproducing

Setup is in the [benchmark's README](#). Every number on this page comes from one of these:

number	command
scores at every length, both question types	<code>python -m kbench.benchmarks.memoryagentbench.run --run_id <id> --lengths 6k,32k,64k,262k</code>
the tables, and how often the answer was retrieved	<code>python -m kbench.benchmarks.memoryagentbench.check <id> --out report.md</code>
the whole-history arm	<code>python -m kbench.benchmarks.memoryagentbench.run_full_context --run_id <id2> --ingest_run_id <id></code>
its score of 85, and its token count	<code>python -m kbench.benchmarks.memoryagentbench.score_partial <parquet> <that run's adapter log></code>